Markdown 技术文档

Markdown 技术文档
1. Markdown 简介
2. 标题（Headings）
一级标题
二级标题
三级标题
四级标题
五级标题
六级标题
3. 段落与换行（Paragraphs and Line Breaks）
4. 文本样式（Text Formatting）
5. 列表（Lists）
5.2 无序列表（Unordered List）
5.3 有序列表（Ordered List）
5.4 嵌套列表（Nested List）
5.5 任务列表（Task List）
5.6 混合列表
5.7 使用建议
6. 代码与语法高亮（Code and Syntax Highlighting）
6.1 行内代码（Inline Code）
6.2 基本代码块（Fenced Code Block）
6.3 语法高亮（Syntax Highlighting）
6.4 显示文件路径或命令行
6.5 嵌套代码块（在列表中使用）
6.6 提示与最佳实践
7. 链接与图片（Links and Images）
7.1 插入超链接（Inline Links）
7.2 链接带标题（Title 属性）
7.3 引用式链接（Reference Links）
7.4 插入图片（Images）
7.5 图片带标题（Title 属性）
7.6 图片引用式写法
7.7 图片与链接混合（点击图片跳转）
7.8 相对路径（本地图片）
7.9 使用建议
8. 表格（Tables）
8.1 基本表格语法
8.2 对齐方式设置
8.3 表格内容格式化
8.4 宽表格的可读性
8.5 HTML 表格的进阶用法
8.6 使用建议
9. 分隔线与引用（Horizontal Rules & Blockquotes）
9.1 分隔线（Horizontal Rules）
9.1.1 基本语法
9.1.2 使用场景
9.2 引用（Blockquotes）
9.2.1 基本语法
9.2.2 嵌套引用（Nested Blockquotes）

9.2.3 引用中包含其他 Markdown 元素
9.2.4 引用与段落分隔
9.3 高级引用样式（自定义提示块）
9.4 使用建议
10. 列表（Lists）
10.1 无序列表（Unordered Lists）
10.1.1 多级无序列表
10.2 有序列表（Ordered Lists）
10.3 混合嵌套列表
10.4 列表中的段落与代码块
10.5 任务列表（Task Lists）
10.6 使用建议
11. 代码与语法高亮（Code & Syntax Highlighting）
11.1 行内代码（Inline Code）
11.2 多行代码块（Fenced Code Blocks）
11.3 指定语言启用语法高亮
常用语言标识
11.4 代码块中的缩进与文本说明
11.5 行内与多行代码结合使用
11.7 高级技巧：带行号、复制按钮（依赖渲染器）
11.8 使用建议
12. 链接引用与锚点跳转（Link References & Anchors）
12.1 链接回顾
12.2 引用式链接（Reference Links）
引用式链接的优点
12.3 自动引用（隐式命名）
12.4 锚点链接（Anchors）
12.4.1 自动生成锚点（常见于 GitHub / Typora）
12.4.2 手动锚点（使用 HTML 方式）
12.4.3 跨文件锚点跳转
12.5 在目录中使用锚点（Table of Contents）
目录
12.6 链接+锚点混合用法
12.7 使用建议
13. Markdown 扩展语法（Extended Markdown Features）
13.1 表情符号（Emoji）
13.2 脚注（Footnotes）
13.3 自动目录（Table of Contents, TOC）
13.4 任务列表（Task Lists）
13.5 删除线（Strikethrough）
13.6 下标与上标
13.7 折叠内容区块（Details / Summary）
13.8 高亮文本（Highlight）
13.9 警示块（Admonition Blocks / 提示框）
13.10 数学公式（LaTeX 支持）
13.11 扩展语法兼容性表
14. Markdown 与 HTML 混用（Mixing Markdown with HTML）
14.1 基本混合原则
14.2 常用 HTML 标签示例
段落与换行
字体样式
14.3 图片排版控制
14.4 分栏布局

14.5 自定义表格样式
14.6 折叠与交互展示
14.7 控制对齐和间距
14.8 内嵌超链接样式增强
14.9 导入外部资源（仅限可控场景）
14.10 渲染器兼容说明
15. Markdown 与表格进阶使用（Advanced Tables in Markdown）
15.1 基本表格语法回顾
15.2 对齐方式控制
15.3 表内使用格式语法
15.4 大表格的换行与多行单元格
15.5 合并单元格（HTML 扩展）
15.6 表格宽度与样式控制
15.7 嵌入图片、链接与代码
15.8 表格与布局结合
15.9 渲染兼容性

---

1. Markdown 简介

1.1 定义
Markdown 是一种纯文本格式的标记语言，由 John Gruber 于 2004 年创建。
其核心目标是让文档作者使用简洁的符号，在保持可读性的同时，实现与 HTML 等排版语言的互通。

1.2 特点

- 语法简单，易于记忆和使用；
- 插入符号明确，可直接阅读；
- 支持多平台渲染，如 GitHub、Quarto、Jupyter Notebook；
- 可通过扩展语法和插件实现更多样的功能（如任务列表、脚注、数学公式）。

1.3 应用场景

- 软件项目的 README 文件；
- 技术博客、使用说明、发布日志；
- Quarto / R Markdown 技术报告与论文；
- API 文档、学术笔记。

1.4 Markdown 与 HTML 的关系
Markdown 的语法设计可以无缝转换为 HTML。例如：

```
**加粗文本**
```

可生成以下 HTML 输出：

```
<strong>加粗文本</strong>
```

好的，下面是第二章的内容：

2. 标题（Headings）

2.1 基本概念

标题用于组织文档结构，方便分层阅读。Markdown 使用井号符号（`#`）定义标题层级，从一级标题到六级标题共有六个层次。

每多一个 `#`，标题层级就会降低一级。

2.2 基本语法

```
# 一级标题
## 二级标题
### 三级标题
#### 四级标题
##### 五级标题
###### 六级标题
```

2.3 渲染结果

一级标题

二级标题

三级标题

四级标题

五级标题

六级标题

2.4 使用规范

1. 一级标题（`#`）通常用于文档标题。
2. 二级、三级标题用于章节和小节的结构化划分。
3. 同层级标题的数量与内容应保持一致，以保证层次清晰。
4. 标题应简洁明了，避免过长描述。

2.5 带锚点的标题

Markdown 渲染器（如 Quarto、GitHub）会自动为每个标题生成锚点，便于内部跳转。

示例：

```
[跳转到结论部分](#15-结论)
```

也可通过手动方式定义锚点：

```
## 数据准备 {#data-preparation}
```

此时可以使用 `[前往数据准备部分](#data-preparation)` 进行页内链接。

2.6 编号标题的写法

在较正式的技术文档或报告中，可以在标题前手动添加编号：

```
## 1. 简介
## 2. 语法结构
## 3. 总结
```

这样在渲染后的文档中标题会自动带编号，更加规范。

---

3. 段落与换行（Paragraphs and Line Breaks）

3.1 段落定义

Markdown 中的段落由一个或多个连续的文本行组成。
段落之间通过**空一行**的方式进行分隔。

示例：

```
这是第一段。

这是第二段。
```

渲染结果：

这是第一段。

这是第二段。

3.2 换行方式

Markdown 默认会忽略单个回车，因此若希望在同一段中实现换行，有两种常用方法：

1. **在行末添加两个空格**
   在当前行末尾添加两个空格后换行。

   ```
   这是第一行␣␣
   这是第二行
   ```

   渲染结果：
   这是第一行
   这是第二行

2. **使用 HTML 的 `<br>` 标签**
   `<br>` 被 Markdown 解析器识别，可强制换行。

   ```
   这是第一行<br>这是第二行
   ```

   渲染结果：
   这是第一行
   这是第二行

3.3 段落缩进

标准 Markdown 不支持段首缩进。若要实现缩进效果，可使用 HTML 实体 ` ` 或 CSS 控制：

```
 这是带缩进的段落。
```

渲染结果：

　这是带缩进的段落。

3.4 空白与换行控制

- Markdown 中的多个空格在渲染时会合并为一个空格；
- 若需展示连续空格或固定宽度排版，可使用代码块或 HTML `<pre>` 元素包裹。

```
这是    三个空格。
```

渲染效果：
这是 三个空格。

```
<pre>这是    三个空格。</pre>
```

渲染效果（保持空格）：

<pre>这是   三个空格。</pre>

3.5 使用建议

1. 每个段落前后留一行空白，增强可读性；
2. 控制段落长度，避免行过长；
3. 对于逻辑相对独立的内容，建议新起一段书写；
4. 在报告或书面格式中，需要更高控制时可混合 HTML。

---

4. 文本样式（Text Formatting）

4.1 基本文本样式

Markdown 提供多种基础文字格式，如**加粗**、*斜体*、~~删除线~~、`行内代码` 等。
下面列出常用语法及其效果。

功能	语法	示例渲染
加粗	`**文本**` 或 `__文本__`	**加粗文本**
斜体	`*文本*` 或 `_文本_`	*斜体文本*
加粗 + 斜体	`***文本***`	***加粗 + 斜体***
~~删除线~~	`~~文本~~`	~~删除线文本~~

功能	语法	示例渲染
`行内代码`	`` `代码` ``	代码

4.2 强调与层次控制

Markdown 的加粗和斜体常用于语义上的区分：

- *斜体*：轻微强调、术语引用或外语文字；
- **加粗**：重点内容、结论或提示；
- `行内代码`：用于展示变量名、指令、文件路径等技术内容；
- ***加粗斜体***：高重要性强调，建议谨慎使用。

4.3 引用（Blockquote）

引用用于插入他人语句、提示信息或说明文字。
使用“`>`”符号开头即可。

```
> 这是一个引用的段落。
```

渲染结果：

> 这是一个引用的段落。

可多层嵌套：

```
> 第一层引用
>> 第二层引用
>>> 第三层引用
```

渲染结果：

> 第一层引用
>
> > 第二层引用
> >
> > > 第三层引用

4.4 强调块（提示性引用）

部分 Markdown 渲染器（如 GitHub、Docsify、Quarto）扩展了引号语法，用于生成提示块。例如：

```
> **注意：** 保存文件前请确认路径正确。
>
> **提示：** 可使用快捷键 `Ctrl + S`。
```

渲染结果：

> **注意：** 保存文件前请确认路径正确。
> **提示：** 可使用快捷键 `Ctrl + S`。

4.5 水平分割线（Horizontal Rule）

用于段落或主题间的视觉分隔。
语法：使用三个或以上的 `-`、`*`、`_` 均可。

```
---
```

渲染结果：

4.6 特殊字符转义

若希望显示 Markdown 保留符号（如 `*`、`#`、`_` 等），需要使用反斜杠 `\` 进行转义。

示例：

```
\*不会被渲染为斜体\*
```

渲染效果：
不会被渲染为斜体

4.7 使用建议

1. 控制格式强调数量，避免视觉层次混乱；
2. 对于代码、公式类文档，优先用 `行内代码` 而非加粗；
3. 在引言或报告中，可结合分割线和引用增强层次性。

---

5. 列表（Lists）

5.1 列表的作用

列表用于组织信息，使内容结构清晰、条理分明。
Markdown 支持两种主要的列表：

- **无序列表（Unordered List）**
- **有序列表（Ordered List）**

此外，还支持 **嵌套列表（Nested List）** 和 **任务列表（Task List）** 等扩展语法。

5.2 无序列表（Unordered List）

语法规则

可使用 `*`、`-` 或 `+` 来创建无序列表，各符号效果相同。

```
- 苹果
- 香蕉
- 橙子
```

渲染结果：

- 苹果
- 香蕉
- 橙子

注意事项：

- 列表项前的符号与内容之间要保留一个空格；
- 同一级别的项目最好使用相同符号，保持一致性。

5.3 有序列表（Ordered List）

语法规则
使用数字加英文句点 `.` 来定义：

```
1. 第一步
2. 第二步
3. 第三步
```

渲染结果：

1. 第一步
2. 第二步
3. 第三步

说明：
Markdown 在渲染时会自动重新编号，
因此以下写法的渲染效果相同：

```
1. 第一项
1. 第二项
1. 第三项
```

5.4 嵌套列表（Nested List）

嵌套列表用于层级内容表达，在子列表前添加 **两个或四个空格**（依解析器不同）即可。

```
- 水果
  - 苹果
  - 香蕉
- 蔬菜
  - 菠菜
  - 西红柿
```

渲染结果：

- 水果
    - 苹果

    - 香蕉
- 蔬菜
    - 菠菜
    - 西红柿

5.5 任务列表（Task List）

任务列表常用于项目管理、计划清单（支持 GitHub、Quarto、Typora 等渲染器）。

语法：

```
- [ ] 未完成任务
- [x] 已完成任务
```

渲染结果：

- [ ] 未完成任务
- [x] 已完成任务

5.6 混合列表

可混合使用有序与无序列表：

```
1. 日程安排
   - 早餐
   - 会议
2. 下午计划
   - 散步
   - 阅读
```

渲染结果：

1. 日程安排
    - 早餐
    - 会议
2. 下午计划
    - 散步
    - 阅读

5.7 使用建议

1. 无序列表适合归纳项；
2. 有序列表适合步骤性内容；
3. 避免层级过深（建议不超过 3 层）；

4. 注意缩进一致，否则渲染可能出现混乱；
5. 在文档说明中，可以合理搭配任务列表使用。

---

6. 代码与语法高亮（Code and Syntax Highlighting）

Markdown 非常适合展示代码示例。
它提供了两种方式来插入代码：

- **行内代码（Inline Code）**
- **代码块（Code Block）**

借助语法高亮功能，还能使程序代码更加清晰。

6.1 行内代码（Inline Code）

当代码或命令仅占一行或一句时，可使用反引号 `` ` `` 包裹。

语法：

```
请使用命令 `git status` 检查仓库状态。
```

渲染结果：
请使用命令 `git status` 检查仓库状态。

技巧：

- 若内容本身包含反引号，可用双反引号包裹：
  `使用 \` 进行转义`

6.2 基本代码块（Fenced Code Block）

使用三个反引号 ```` ```` 或三个波浪线 ~~~ 创建代码块。

语法：

```
```
print("Hello, Markdown!")
```
```

渲染结果：

```
print("Hello, Markdown!")
```

6.3 语法高亮（Syntax Highlighting）

在代码块的开头三个反引号后标注语言类型（如 `python`、`java`、`bash`），Markdown 渲染器将自动应用语法高亮。

示例：

```
```python
def greet(name):
    print(f"Hello, {name}!")
```
```

渲染结果：

```python
def greet(name):
    print(f"Hello, {name}!")
```

6.4 显示文件路径或命令行

行内代码也常用于展示文件路径与命令：

```
请打开文件 `config.yml` 并修改参数。
```

渲染结果：
请打开文件 `config.yml` 并修改参数。

命令行示例：

```
npm install markdown-it
```

6.5 嵌套代码块（在列表中使用）

当代码块在列表中时，需要额外缩进至少 4 个空格：

```
1. 运行以下代码：

    ```bash
    python main.py
```

```
**渲染结果：**

1. 运行以下代码：

    ```bash
    python main.py
```

6.6 提示与最佳实践

项目	建议做法
文件名或命令	使用 `行内代码` 包裹
长代码	使用三反引号多行代码块
多语言展示	指明语言类型实现语法高亮
格式稳定	三反引号前后必须独占一行

---

7. 链接与图片（Links and Images）

Markdown 可以轻松插入 **超链接** 与 **图片**。
这两者语法非常相似，区别仅在于图片前多一个感叹号 `!`。

7.1 插入超链接（Inline Links）

基本语法：

```
[显示文字](链接地址)
```

示例：

```
[访问 Markdown 官方教程](https://www.markdownguide.org)
```

渲染结果：
[访问 Markdown 官方教程]

7.2 链接带标题（Title 属性）

可在链接中添加提示文字（鼠标悬停时显示）。

```
[GitHub](https://github.com "访问 GitHub 官网")
```

渲染结果：
[GitHub]

7.3 引用式链接（Reference Links）

当同一链接多次出现时，可使用引用式写法以减少重复。

语法：

```
[Markdown 官网][md]
[GitHub 官网][gh]

[md]: https://www.markdownguide.org
[gh]: https://github.com
```

渲染效果：
[Markdown 官网]
[GitHub 官网]

7.4 插入图片（Images）

图片语法几乎与链接相同，只是前面加了 `!`。

```
![替代文字](图片URL)
```

示例：

```
![Markdown 图标](https://markdown-here.com/img/icon256.png)
```

渲染结果：

7.5 图片带标题（Title 属性）

同样可为图片添加提示文字：

```
![GitHub Logo](https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png "GitHub 标志")
```

渲染结果：

7.6 图片引用式写法

```
![Alt 文本][logo]

[logo]: https://markdown-here.com/img/icon256.png "Markdown 图标"
```

渲染结果：

7.7 图片与链接混合（点击图片跳转）

可以让图片本身成为链接：

```
[![Logo](https://markdown-here.com/img/icon256.png)](https://www.markdownguide.org)
```

渲染结果：

7.8 相对路径（本地图片）

若图片位于同一目录下，可使用相对路径引用：

```
![封面图片](./images/cover.png)
```

> **说明**：在 GitHub、Docsify、Obsidian 等平台，本地图片需要与 Markdown 文件位于同一仓库或工作区中，否则不会显示。

7.9 使用建议

场景	推荐做法
外部网页	使用标准超链接语法
多次使用同一链接	使用引用式链接
图片说明	添加替代文字（Alt）与标题
图片过大	可使用 HTML 标签 `<img width="300" />` 控制大小
点击图片跳转	图片嵌入链接的组合语法

---

8. 表格（Tables）

表格用于展示结构化数据，是 Markdown 中非常实用的语法之一。
Markdown 的表格语法简洁直观，适合展示参数、对比、计划等内容。

8.1 基本表格语法

表格通过 **竖线** `|` 分隔列，使用 **短横线** `-` 区分表头与表体。

语法示例：

```
| 名称 | 类型 | 说明 |
|------|------|------|
| 苹果 | 水果 | 富含维C |
| 菠菜 | 蔬菜 | 富含铁质 |
```

渲染结果：

名称	类型	说明
苹果	水果	富含维C
菠菜	蔬菜	富含铁质

8.2 对齐方式设置

可以通过在分隔线中加 **冒号** `:` 来控制文本对齐：

- `:---`：左对齐
- `:---:`：居中对齐
- `---:`：右对齐

示例：

```
| 项目 | 数量 | 价格 |
|:-----|:----:|-----:|
| 苹果 | 10   | 5.00 |
| 香蕉 | 6    | 3.50 |
```

渲染结果：

项目	数量	价格
苹果	10	5.00
香蕉	6	3.50

8.3 表格内容格式化

表格中的内容可以包含文字、行内代码、甚至是 emoji。

示例：

```
| 功能 | 是否支持 | 备注 |
|------|----------|------|
| 加粗文字 | x | 使用 `**bold**` |
| 斜体文字 | x | 使用 `*italic*` |
| 超链接 | x | `[Example](#)` |
```

渲染结果：

功能	是否支持	备注
加粗文字	x	使用 `**bold**`
斜体文字	x	使用 `*italic*`
超链接	x	`[Example](#)`

8.4 宽表格的可读性

- 若列过多，可在代码中对齐竖线，使其整齐；
- 在部分渲染器中（如 VS Code Markdown 预览），可以自动识别对齐；
- 建议保持 3 ~ 8 列以内，易于阅读。

例：

```
| 年份 | 销售额（万元） | 同比增长 | 重要事项 |
|------|----------------|----------|----------|
| 2023 | 380 | +12% | 推出新产品 |
| 2024 | 420 | +10% | 拓展海外市场 |
| 2025 | 465 | +11% | 自动化升级完成 |
```

渲染结果：

年份	销售额 (万元)	同比增长	重要事项
2023	380	+12%	推出新产品
2024	420	+10%	拓展海外市场
2025	465	+11%	自动化升级完成

8.5 HTML 表格的进阶用法

Markdown 不支持合并单元格（rowspan / colspan）。
如果需要更复杂的布局，可直接使用 HTML 语法：

```
<table>
  <tr>
    <th>分类</th>
    <th>项目</th>
    <th>价格</th>
  </tr>
  <tr>
    <td rowspan="2">水果</td>
    <td>苹果</td>
    <td>¥5</td>
  </tr>
  <tr>
    <td>香蕉</td>
    <td>¥3</td>
  </tr>
</table>
```

渲染结果：

分类	项目	价格
水果	苹果	¥5
	香蕉	¥3

8.6 使用建议

场景	建议方案
简单数据展示	推荐使用 Markdown 原生表格语法
需要单元格合并	使用嵌入式 HTML 表格
对齐要求较高	调整冒号位置或使用渲染器自动格式化

场景	建议方案
多平台兼容	尽量避免复杂 HTML 标签

---

9. 分隔线与引用（Horizontal Rules & Blockquotes）

Markdown 提供了简洁的方式来为文本添加视觉结构与引用内容。
本章将介绍 **分隔线（Horizontal Rules）** 与 **引用块（Blockquotes）** 的使用方法。

9.1 分隔线（Horizontal Rules）

分隔线用于在内容之间创建视觉上的分割。
它能让章节或主题更加清晰地分段。

9.1.1 基本语法

可使用以下任意语法创建分隔线：

```
---

***
___
```

渲染结果：

> **提示**：
> 分隔线前后应留出一行空白，才能被正确识别。

9.1.2 使用场景

场景	示例
划分章节	将文档的两个主题分隔开
视觉强调	分隔摘要与正文内容
可视排版	在简历或报告中增加层次感

9.2 引用（Blockquotes）

引用块用于展示引用的文字、引用他人言论、或作为提示块。

9.2.1 基本语法

每一行以 `>` 开头，即可生成引用：

```
> 坚持不是胜利，
> 坚持本身就是一种胜利。
```

渲染结果：

> 坚持不是胜利，
> 坚持本身就是一种胜利。

9.2.2 嵌套引用（Nested Blockquotes）

多层引用可通过增加 `>` 的数量来实现：

```
> 第一层引用
>> 第二层引用
>>> 第三层引用
```

渲染结果：

> 第一层引用
>
> > 第二层引用
> >
> > > 第三层引用

9.2.3 引用中包含其他 Markdown 元素

引用块可以与 **标题、列表、代码块** 等语法混合使用。

示例：

```
> ### 引用标题
> - 列表项 A
> - 列表项 B
>
> ```bash
> echo "Hello Markdown!"
> ```
```

渲染结果：

> ## 引用标题
>
> - 列表项 A
> - 列表项 B
>
> ```
> echo "Hello Markdown!"
> ```

9.2.4 引用与段落分隔

引用中可以包含多个段落，段落之间需用空行分隔：

```
> 这是第一段。
>
> 这是引用块中的第二段。
```

渲染结果：

> 这是第一段。
>
> 这是引用块中的第二段。

9.3 高级引用样式（自定义提示块）

许多 Markdown 渲染器支持对引用块样式进行扩展。
可用引用块充当提示、警告、信息框等。

例如在 GitHub、Docsify、Obsidian 中：

```
> **提示：** 你可以在引用中使用 Emoji 来增强视觉提示。
>
> 使用 `>` 开头的多行文字将自动组合为一个高亮信息块。
```

渲染效果：

> **提示：** 你可以在引用中使用 Emoji 来增强视觉提示。
> 使用 `>` 开头的多行文字将自动组合为一个高亮信息块。

9.4 使用建议

类型	推荐用途
分隔线 `---`	分割文章部分内容
引用块 `>`	引用信息、说明文字、名言
嵌套引用	层级讨论或对话样式
扩展引用（带图标）	用于提示、警告、重点提示

10. 列表（Lists）

Markdown 提供了三种常用的列表类型：

- **无序列表（Unordered Lists）**
- **有序列表（Ordered Lists）**
- **任务列表（Task Lists）**（GitHub 等平台扩展功能）

列表能帮助你清晰地组织信息、展示步骤、或进行要点归纳。

10.1 无序列表（Unordered Lists）

无序列表用于展示没有顺序的项目。
可以用 `-`、`*` 或 `+` 来创建，效果相同。

语法示例：

```
- 苹果
- 香蕉
- 樱桃
```

渲染结果：

- 苹果
- 香蕉
- 樱桃

10.1.1 多级无序列表

在每一级项目前加两个或四个空格，即可创建嵌套列表。

```
- 水果
  - 苹果
  - 香蕉
- 蔬菜
  - 菠菜
  - 胡萝卜
```

渲染结果：

- 水果
  - 苹果
  - 香蕉
- 蔬菜
  - 菠菜
  - 胡萝卜

10.2 有序列表（Ordered Lists）

有序列表用于展示有步骤或顺序的内容。
语法使用数字加英文句点，如 `1.`、`2.`。

```
1. 打开电脑
2. 启动编辑器
3. 编写代码
```

渲染结果：

1. 打开电脑
2. 启动编辑器
3. 编写代码

> **提示：**
> 数字部分不会影响显示顺序——你可以全部使用 `1.`，Markdown 依然会自动编号。

例如：

```
1. 苹果
1. 香蕉
1. 樱桃
```

渲染结果仍为：

1. 苹果
2. 香蕉
3. 樱桃

10.3 混合嵌套列表

你可以混合使用有序与无序列表：

```
1. 水果
   - 苹果
   - 橙子
2. 蔬菜
   - 菠菜
   - 胡萝卜
```

渲染结果：

1. 水果
    - 苹果
    - 橙子
2. 蔬菜

    - 菠菜
    - 胡萝卜

10.4 列表中的段落与代码块

列表中可以包含多行文字、代码块，或其他 Markdown 元素。
子内容需缩进至少 4 个空格（或 1 个制表位）。

示例：

```
1. 安装步骤：

   执行以下命令：
   ```bash
   npm install
```

2. 启动程序：

   ```
   npm start
```

```
**渲染结果：**

1. 安装步骤：

   执行以下命令：
   ```bash
   npm install
```

2. 启动程序：

   ```
   npm start
```

10.5 任务列表（Task Lists）

任务列表是一种 **GitHub Flavored Markdown (GFM)** 扩展语法。
可用于待办事项或进度展示。

语法示例：

```
- [x] 完成第1章
- [x] 完成第2章
- [ ] 撰写第3章
```

渲染结果（在支持的平台中，如 GitHub、Obsidian）：

- ☑ 完成第1章
- ☑ 完成第2章

- [ ] 撰写第3章

10.6 使用建议

类型	用途	特点
无序列表 `-`	列出要点	简洁、灵活
有序列表 `1.`	展示步骤	自动编号、层级清晰
混合嵌套	分类说明	结构化展示
任务列表 `[ ]`	待办清单	可交互勾选（部分平台）

> 建议：
> 在写规范文档、操作指南、笔记记录时，合理地使用不同类型的列表，既能让阅读逻辑清晰，又能增强结构层次。

---

11. 代码与语法高亮（Code & Syntax Highlighting）

Markdown 不仅适合写文档与笔记，也非常适合记录 **代码**。
本章将介绍如何在 Markdown 中插入单行代码、多行代码块，并为不同语言启用**语法高亮**。

11.1 行内代码（Inline Code）

用于在段落中展示简短的代码或命令。

语法：
使用 **反引号** ```（位于键盘左上角 Esc 下方）括起来。

```
请执行命令 `npm install` 进行安装。
```

渲染结果：
请执行命令 `npm install` 进行安装。

> 提示：
> 行内代码不会触发高亮，但会采用固定宽度字体（monospace），更易区分。

11.2 多行代码块（Fenced Code Blocks）

用于展示多行代码或完整函数。
以 **三个反引号** ```` ```` 开头和结束。

```
console.log("Hello Markdown");
```

渲染结果：

```
console.log("Hello Markdown");
```

11.3 指定语言启用语法高亮

在代码块的起始反引号后写上语言名（例如 `js`、`python`、`html` 等），
Markdown 渲染器会自动为该语言添加颜色高亮。

```python
def greet(name):
    print(f"Hello, {name}!")
```

```
**渲染结果（带语法高亮）：**

```python
def greet(name):
    print(f"Hello, {name}!")
```
```

常用语言标识

语言	标识	示例
JavaScript	`js` / `javascript`	```js
Python	`python`	```python
Bash / Shell	`bash` / `shell` / `sh`	```bash
JSON	`json`	```json
HTML	`html`	```html
CSS	`css`	```css
C / C++	`c`, `cpp`	```cpp
Java	`java`	```java

11.4 代码块中的缩进与文本说明

代码块中可嵌入注释、输出结果或说明。

```bash
# 安装依赖
npm install

# 启动项目
npm start
```

```
**渲染结果：**

```bash
# 安装依赖
npm install

# 启动项目
npm start
```
```

11.5 行内与多行代码结合使用

在写教程时常结合使用：

```
请运行命令 `python main.py`，
或在代码块中添加完整内容：

```python
if __name__ == "__main__":
    print("Running script...")
```
```

```
---

### 11.6 代码块中的特殊字符

如果你需要在 Markdown 里展示 Markdown 自身语法（例如展示 `#`、`*` 等符号），
务必使用代码块，否则会被解析为格式。

例如：

```markdown
\# 这是一个不作为标题的井号
* 这不是斜体
```
```

渲染后仍为：

这是一个不作为标题的井号
* 这不是斜体

11.7 高级技巧：带行号、复制按钮（依赖渲染器）

一些现代 Markdown 渲染器（如 **VS Code Preview**、**Obsidian**、**Docsify** 或 **GitBook**）
支持更多特性：

功能	支持方式	示例
行号	```{.language-numbered} 或启用插件	展示代码行号
可复制按钮	GitBook / Docsify 内置	自动出现 Copy 按钮
行高亮	```js{1,3}	高亮第1行与第3行

这些属于 **扩展功能**，并非所有平台都支持。

11.8 使用建议

场景	推荐语法	示例
命令或变量说明	行内代码	`var name = "Tom"`
展示函数或配置	三反引号代码块	`python ...`
写技术笔记	带语言名的代码块	`bash npm install`
显示 Markdown 语法	使用 `\` 转义	`\*text\*`

> **总结：**
>
> - 行内代码：突出关键命令
> - 多行代码块：展示完整逻辑
> - 指定语言名：启用语法高亮
> - 使用反引号，而非引号或单引号

---

12. 链接引用与锚点跳转（Link References & Anchors）

Markdown 除了支持一般的超链接（如第5章提到的 `[文字](链接)`），
还可以实现 **引用式链接管理** 与 **文档内锚点跳转**，
这对于编写长文档或知识手册非常实用。

12.1 链接回顾

基础链接语法如下：

```
[前往百度](https://www.baidu.com)
```

渲染结果：
[前往百度](https://www.baidu.com)

12.2 引用式链接（Reference Links）

当同一个链接被多次使用时，建议采用“**引用声明**”方式：
这样能让正文更简洁，方便统一修改。

语法结构：

```
这是一个 [示例链接][example]。

[example]: https://www.example.com
```

渲染结果：

这是一个 [示例链接][example]。

> 它在渲染时等价于：
> `[示例链接](https://www.example.com)`

引用式链接的优点

普通写法	引用式写法
`[GitHub](https://github.com)`	`[GitHub][gh]` `[gh]: https://github.com`
链接杂乱	正文更清晰、更易维护
修改需逐一改	修改一次即可生效

12.3 自动引用（隐式命名）

如果你不想显式写 `[example]: ...` 名称，也可以省略。

语法：

```
[Google][]

[Google]: https://www.google.com
```

等价于：

```
[Google](https://www.google.com)
```

> 这种写法在大型文档或书籍项目里尤为常见，
> 可以把所有链接集中到文件底部统一管理。

12.4 锚点链接（Anchors）

锚点用于 **页面内跳转**，
当你的 Markdown 文档较长时，可以快速定位到某个章节。

12.4.1 自动生成锚点（常见于 GitHub / Typora）

GitHub 等渲染器会自动为标题生成锚点链接，规则是：

> 转成小写 → 去除标点 → 空格变为 `-`

例如：

标题	自动生成锚点
`## 使用方法`	`#使用方法` → `#使用方法`
`### Getting Started`	`#getting-started`
`### 第3章：安装步骤`	`#第3章安装步骤`

示例：

```
[跳转到安装步骤](#第3章安装步骤)
```

点击后会自动定位到文档中该标题位置。

12.4.2 手动锚点（使用 HTML 方式）

Markdown 也支持 HTML 语法，可手动定义锚点：

```
<a name="part1"></a>
## 第一部分

跳转到 → [第一部分](#part1)
```

这样可完全自定义锚点名。

12.4.3 跨文件锚点跳转

可同时指定文档路径和锚点，实现跨文件跳转：

```
[查看配置说明](./config.md#配置文件)
```

适用于知识库或项目文档体系。

12.5 在目录中使用锚点（Table of Contents）

你可以通过锚点手动创建一个目录（TOC）：

```
## 目录
- [简介](#简介)
- [安装](#安装)
- [使用说明](#使用说明)
```

渲染效果（可点击跳转）：

目录

- 简介
- 安装
- 使用说明

12.6 链接+锚点混合用法

Markdown 链接可以同时包含 **网页 URL** 与 **锚点**。

```
[查看第3节](https://example.com/docs#section3)
```

这会直接跳到网页中的该小节位置。

12.7 使用建议

功能	推荐用法	示例
普通外链	`[名称](URL)`	`[百度](https://baidu.com)`
多次复用链接	引用式链接	`[GitHub][gh]`
单页跳转	锚点链接	`[返回顶部](#目录)`
跨文件跳转	结合路径与锚点	`[下一章](chapter13.md#进阶扩展)`

> **总结：**

> - 链接引用：清爽、统一、易维护
> - 锚点链接：快速跳转，提高文档可导航性
> - 两者结合，让你的 Markdown 文档像网页一样灵活！

太棒了！我们继续 Markdown 系列教程的——

---

13. Markdown 扩展语法（Extended Markdown Features）

标准 Markdown 语法简单高效，但有时功能略显不足。
许多编辑器（如 **Typora**、**Obsidian**、**GitBook**、**VS Code Preview** 等）
在标准语法基础上增加了更丰富的“**扩展语法**”。

本章将带你系统了解这些增强功能。 👇

13.1 表情符号（Emoji）

有些 Markdown 渲染器支持 **Emoji 代码**，
常用于语气表达或轻量提示。

语法：

```
:smile: :rocket: :fire: :heart:
```

渲染效果：
😄 🚀 🔥 ❤️

> 💡 **提示**：
> 不同平台对表情支持不完全一致，
> GitHub / Typora / Obsidian 原生支持 `:emoji:` 格式。
> 如果不生效，也可直接输入系统表情符号 。

13.2 脚注（Footnotes）

脚注能在文中添加注解或解释，
适用于需要补充说明或引用来源的场景。

语法：

```
这是一个带脚注的句子[^1]。

[^1]: 这是脚注的内容。
```

渲染结果：
这是一个带脚注的句子[1]。

> 不同 Markdown 引擎（如 Kramdown、Pandoc）对脚注支持不同，
> 建议在 **Typora / Obsidian / GitBook** 中使用效果最佳。

13.3 自动目录（Table of Contents, TOC）

部分编辑器支持通过标记自动生成目录，
能根据文档标题（`#`，`##`，`###`）自动生成可点击的索引。

语法示例（Typora 支持）：

```
[TOC]
```

渲染结果：

会根据你当前文档所有标题生成一个自动更新的目录。

> ✅ **优点**：文档结构更新后目录自动变化。
> ⚠️ **注意**：GitHub 原生渲染不识别 `[TOC]`。

13.4 任务列表（Task Lists）

在 **GitHub Markdown** 中非常常见，
可以用来记录待办（To-Do）或计划事项。

语法：

```
- [x] 已完成任务
- [ ] 待办任务
- [ ] 审核更新
```

效果：

- [x] 已完成任务
- [ ] 待办任务
- [ ] 审核更新

> 💡 可以嵌套使用，像项目计划表那样排布。
> 在 GitHub / 笔记软件中甚至可以直接点击复选框。

13.5 删除线（Strikethrough）

删除线用于标记废弃、修改或过期的内容。

语法：

```
这是~~删除的文本~~。
```

效果：
这是~~删除的文本~~。

13.6 下标与上标

某些 Markdown 扩展支持化学式、公式或学术标注。

语法（部分渲染器支持）：

```
H~2~O
E = mc^2^
```

效果：
H~2~O
E = mc^2^

13.7 折叠内容区块（Details / Summary）

可将长段文字折叠成可展开的“说明块”，常用于 FAQ、补充说明。

语法（需支持 HTML 渲染）：

```
<details>
<summary>点击展开查看详情</summary>

这里是隐藏的文字内容，可以放代码、图片或说明。

</details>
```

渲染结果：

▶ 点击展开查看详情

13.8 高亮文本（Highlight）

用来强调重点内容。部分 Markdown 方言支持 `==高亮==`。

语法：

```
这是一句 ==重要的提示==。
```

效果（若支持）：
这是一句 ==重要的提示==。

> ⚠️ 如果渲染器不支持，可用 **加粗** `**...**` 替代。

13.9 警示块（Admonition Blocks / 提示框）

在技术文档或教程中常用的“提示”“警告”“注意”框。
某些平台（如 GitBook、Obsidian、Docsify）支持以下格式：

```
> [!NOTE]
> 这是一个提示信息。

> [!WARNING]
> 请小心该操作可能有风险！

> [!TIP]
> 你也可以自定义图标与样式。
```

渲染后效果（在支持的平台中）：

> **Note**
> 这是一个提示信息。

> **Warning**
> 请小心该操作可能有风险！

> **Tip**
> 你也可以自定义图标与样式。

13.10 数学公式（LaTeX 支持）

对于技术文档或学术论文，可用 LaTeX 语法编写公式。

行内公式：

```
这是行内公式：$E = mc^2$。
```

独立公式：

```
$$
\int_a^b f(x) dx = F(b) - F(a)
$$
```

效果（若渲染器支持）：

这是行内公式：$E = mc^2$

$$\int_a^b f(x) dx = F(b) - F(a)$$

> 支持平台：Typora、Obsidian、Jupyter、GitBook、MkDocs（启用 MathJax 或 KaTeX）

13.11 扩展语法兼容性表

功能	标准 Markdown	GitHub	Typora	Obsidian / GitBook
Emoji	NO	YES	YES	YES

功能	标准 Markdown	GitHub	Typora	Obsidian / GitBook
脚注	NO	YES	YES	YES
自动目录 `[TOC]`	NO	NO	YES	YES
任务列表	YES	YES	YES	YES
折叠区块 `<details>`	YES（HTML）	YES	YES	YES
数学公式	NO	NO（需插件）	YES	YES
高亮 `==text==`	NO	NO	YES	YES
提示框 `[!NOTE]`	NO	NO	YES	YES

---

14. Markdown 与 HTML 混用（Mixing Markdown with HTML）

Markdown 本身是一种轻量级标记语言，用于快速编写格式化文本。
但有时标准语法无法满足排版、样式等更复杂的需求，此时可以**直接嵌入 HTML**。
Markdown 渲染引擎通常内置 HTML 解析器，可以混合使用两者。

14.1 基本混合原则

Markdown 会自动识别 HTML 标签。
在文中插入 HTML 时，无需额外转义，直接写即可。

示例：

```
这是段落。

<p>这是一段使用 HTML 写的文字，可以控制样式。</p>
```

渲染后：

这是段落。

这是一段使用 HTML 写的文字，可以控制样式。

14.2 常用 HTML 标签示例

段落与换行

```
<p>这是一个段落。</p>
<br>
<p>这是第二个段落。</p>
```

字体样式

```
<b>加粗文字</b>
<i>斜体文字</i>
<u>下划线</u>
<span style="color:red;">红色文字</span>
<span style="font-size:18px;">大号字体</span>
```

渲染效果：

加粗文字
斜体文字
下划线
红色文字
大号字体

14.3 图片排版控制

Markdown 插入图片后无法设置宽度、对齐方式，
此时可使用 HTML 语法。

```
<img src="example.jpg" width="300" style="display:block;margin:auto;">
```

这可实现居中显示并调整尺寸。

你也可以用 `<figure>` 标签包裹图像与说明：

```
<figure>
  <img src="diagram.png" width="400">
  <figcaption>图 1：系统架构示意图</figcaption>
</figure>
```

14.4 分栏布局

Markdown 不支持多列布局，可以用 `<div>` 控制。

```
<div style="display:flex; gap:20px;">
  <div style="flex:1;">
    <h4>左列</h4>
    <p>这里可以放笔记内容。</p>
  </div>
  <div style="flex:1;">
    <h4>右列</h4>
    <p>这里可以展示代码或图片。</p>
  </div>
</div>
```

这段 HTML 在支持内联样式的渲染器中会呈现为两栏布局。

14.5 自定义表格样式

标准 Markdown 表格版式单一，可以通过 HTML 定义更复杂结构：

```
<table border="1" style="border-collapse:collapse;width:100%;">
  <tr>
    <th>名称</th>
    <th>描述</th>
    <th>备注</th>
  </tr>
  <tr>
    <td>变量A</td>
    <td>控制输入</td>
    <td>需初始化</td>
  </tr>
  <tr>
    <td>变量B</td>
    <td>处理逻辑</td>
    <td>可选参数</td>
  </tr>
</table>
```

这种方式在 GitBook、Obsidian、浏览器端渲染中都有效。

14.6 折叠与交互展示

HTML 标签 `<details>` 和 `<summary>` 可在 Markdown 中直接使用。

```
<details>
  <summary>点击展开查看代码</summary>
  <pre><code>
def hello():
    print("Hello World")
  </code></pre>
</details>
```

适合展示补充说明或额外内容。

14.7 控制对齐和间距

用 `<div>` 的 `align` 属性实现文字对齐：

```
<div align="center">居中内容</div>
<div align="right">右对齐内容</div>
```

自定义间距可使用 `margin` 和 `padding`：

```
<div style="margin-top:20px; padding:10px; background:#f6f6f6;">
这是带背景的区块。
</div>
```

14.8 内嵌超链接样式增强

```
<a href="https://example.com" target="_blank" style="color:#0077cc;text-decoration:none;">
访问外部网站
</a>
```

可通过 `target="_blank"` 指定新标签页打开。

14.9 导入外部资源（仅限可控场景）

HTML 允许在 Markdown 中导入外部内容，例如本地音频、视频。

```
<video src="demo.mp4" width="400" controls></video>

<audio controls>
  <source src="sound.mp3" type="audio/mpeg">
</audio>
```

这种方式适用于离线文档或教程。

14.10 渲染器兼容说明

渲染器	HTML 支持	特别说明
GitHub README	支持大部分 HTML 标签，限制 `script` 等安全风险标签	
Obsidian	基本支持 HTML 与样式属性	
Typora	支持完全 HTML 渲染	
GitBook	支持 HTML + CSS 渲染	
VS Code Markdown Preview	支持基本 HTML，部分样式受限	

---

15. Markdown 与表格进阶使用（Advanced Tables in Markdown）

标准 Markdown 表格语法功能有限，只能用于基本的行列对齐展示。
本章将介绍如何在不同环境下实现更复杂的表格排版与样式控制。

15.1 基本表格语法回顾

标准语法示例：

```
| 项目 | 数量 | 备注 |
|------|------|------|
| 苹果 | 3 | 新鲜 |
| 香蕉 | 5 | 成熟 |
| 橙子 | 2 | 略酸 |
```

效果：

项目	数量	备注
苹果	3	新鲜
香蕉	5	成熟
橙子	2	略酸

> Markdown 表格会自动按竖线 `|` 对齐。
> `:` 号可以控制对齐方式。

15.2 对齐方式控制

在分隔线中使用 `:` 可以设定文字对齐：

```
| 左对齐 | 居中 | 右对齐 |
|:--------|:------:|------:|
| 文本1 | 文本2 | 文本3 |
```

效果：

左对齐	居中	右对齐
文本1	文本2	文本3

15.3 表内使用格式语法

Markdown 表格单元格内仍然支持基本语法，例如：

```
| 功能 | 说明 | 链接 |
|------|------|------|
| **加粗** | `代码` | [跳转](https://example.com) |
```

效果：

功能	说明	链接
加粗	`代码`	跳转

15.4 大表格的换行与多行单元格

标准 Markdown 表格不支持多行内容，
但你可以使用 HTML 来扩展单元格格式。

```
<table>
  <tr>
    <th>章节</th>
    <th>内容说明</th>
  </tr>
  <tr>
    <td>第1章</td>
    <td>介绍基本语法<br>包含标题、段落、列表等</td>
  </tr>
  <tr>
    <td>第2章</td>
    <td>扩展语法<br>链接、图片、代码块</td>
  </tr>
</table>
```

效果：

章节	内容说明
第1章	介绍基本语法 包含标题、段落、列表等
第2章	扩展语法 链接、图片、代码块

15.5 合并单元格（HTML 扩展）

Markdown 无法直接合并单元格，需要使用 HTML：

```
<table border="1">
  <tr>
    <th rowspan="2">类别</th>
    <th colspan="2">统计</th>
  </tr>
  <tr>
    <th>数量</th>
    <th>比例</th>
  </tr>
  <tr>
    <td>水果</td>
```

```
    <td>10</td>
    <td>50%</td>
  </tr>
  <tr>
    <td>蔬菜</td>
    <td>10</td>
    <td>50%</td>
  </tr>
</table>
```

效果：

类别	统计	
	数量	比例
水果	10	50%
蔬菜	10	50%

15.6 表格宽度与样式控制

可以用内联 CSS 控制表格外观：

```
<table style="width:100%; border-collapse:collapse;" border="1">
  <tr style="background:#f0f0f0;">
    <th style="width:20%">名称</th>
    <th>描述</th>
  </tr>
  <tr>
    <td>示例A</td>
    <td>说明文字。</td>
  </tr>
  <tr>
    <td>示例B</td>
    <td>补充说明。</td>
  </tr>
</table>
```

效果：

名称	描述
示例A	说明文字。
示例B	补充说明。

15.7 嵌入图片、链接与代码

表格中可以混排各种内容：

```
| 图示 | 描述 | 链接 |
|------|------|------|
| ![](image.png) | 插入图片 | [查看详情](#) |
| `print("Hello")` | 代码示例 | [更多](https://example.com) |
```

效果：

图示	描述	链接
	插入图片	查看详情
`print("Hello")`	代码示例	更多

15.8 表格与布局结合

可以用表格实现临时布局结构：

```
<table style="width:100%;">
  <tr>
    <td style="width:50%; vertical-align:top;">
      <h4>左侧内容</h4>
      <p>用于展示文字介绍。</p>
    </td>
    <td style="width:50%; vertical-align:top;">
      <h4>右侧内容</h4>
      <pre><code>print("示例")</code></pre>
    </td>
  </tr>
</table>
```

效果：

左侧内容 用于展示文字介绍。	**右侧内容** `print("示例")`

15.9 渲染兼容性

编辑器 / 平台	支持基础表格	支持 HTML 扩展	支持样式
GitHub	YES	部分	限制内联样式
Typora	YES	YES	YES

编辑器 / 平台	支持基础表格	支持 HTML 扩展	支持样式
Obsidian	YES	YES	YES
GitBook	YES	YES	YES
VS Code 预览	YES	YES	部分

---

1. 这是脚注的内容。 ↩